Information Retrieval

Programming Assignment III

Learning to Rank Documents with Boolean Zones and a Linear SVM

for the Cranfield Aeronautics Corpus

Group: search-o-SAURS

Name	Roll Number
Utsa Ghosh	23CS10090
Antariksh Das	23CS10086
Raktim Kundu	23CS30044
Souhardya Dandapat	23CS30054
Sharanya Baidya	23CS30064

Project Repository

https://github.com/utsaghosh04/search-o-SAURS/tree/main/IR_P3

Contents

1 Problem statement and approach **3**
1.1 Implementation choices . . . 3
1.2 Comparisons in this report . . . 3

2 Collection, judgments and query identifiers **3**
2.1 Relevance conversion . . . 4
2.2 Aligning queries with judgments . . . 4

3 Implementation overview **5**
3.1 Learning the two ranking functions . . . 6
3.2 Applying the learned scores . . . 6
3.3 Keeping the experiment reproducible . . . 6

4 Parsing, preprocessing and corpus statistics **6**
4.1 Shared tokenization . . . 6
4.2 Two views of a document . . . 7
4.3 Query-independent statistics and edge cases . . . 7

5 Method A: Boolean field matches and ranking **7**
5.1 Matching rule . . . 7
5.2 Example calculation . . . 8
5.3 Limits of Boolean field matching . . . 8

6 Learning the zone weights **8**
6.1 Choosing the returned list length . . . 8
6.2 Avoiding repeated ranking work . . . 8
6.3 Observed limitation . . . 9

7 SVM features: query coverage and raw frequencies **9**
7.1 Worked frequency example . . . 10

8 SVM features: normalization and retrieval scores **10**
8.1 Cosine similarity . . . 10
8.2 BM25 . . . 11

9 Linear SVM training and exported weights **11**

10 Experiment plan and evaluation **12**
10.1 Precision, recall and F1 . . . 13

11 Test results and discussion **13**
11.1 Comparing identical cutoffs . . . 14
11.2 Why the two models differ . . . 15
11.3 Limits of the result and follow-up experiments . . . 15

12 A judged test-query example **15**
12.1 Method B: all five returned documents . . . 15
12.2 Method A: first five of its 30 returned documents . . . 16

12.3 Reading the example 16

13 A new-query result and the output format **16**
13.1 SVM output 16
13.2 Boolean-zone output 17
13.3 Empty or entirely unknown requests 17

14 Baseline comparison on development queries **17**
14.1 Paired differences and uncertainty 18

15 Regression checks and computational cost **18**
15.1 Full reproduction 19
15.2 Time and memory 19

16 Running the programs **19**
16.1 Install the training dependency 20
16.2 Reproduce the main experiment 20
16.3 Run the consolidated tests 20
16.4 Run the additional validation study 20

17 Running queries and reading the results **20**
17.1 Batch files 21
17.2 Evaluate saved test queries 21
17.3 Use from Python or load an alternate experiment 21

1 Problem statement and approach

Programming Assignment III asks us to learn two ranking functions for the Cranfield collection. Method A learns weights for Boolean matches in the title, author, bibliography and body. Method B learns a linear SVM using the 22 specified query-document features. We evaluate the ranked lists using precision, recall and F1, and retain the learned parameters so that the same models can answer further queries.

Both methods use the same collection and query partitions. Method A reduces each query-document pair to four field matches and learns a weighted sum. Method B uses term coverage, frequency summaries and retrieval scores as inputs to a linear classifier. Documents are ordered by the resulting score. This gives a direct comparison between a compact field-based representation and a richer feature vector.

Training and prediction are separate. We first fit the models using judged queries and select their operating settings on validation queries. Prediction then loads those parameters and ranks the collection for a new request. Judgments are needed only if the returned list is to be evaluated.

1.1 Implementation choices

We use scikit-learn to fit the linear SVM, as permitted by PA3. Record parsing, tokenization, feature calculation, scaling, grid search, ranking and evaluation use the Python standard library. Both models use lowercase alphanumeric tokens without stemming or stopword removal.

The problem statement names the features but leaves their formulas, preprocessing, query split and retrieval cutoffs open. We specify these choices in the report and use them consistently in training and prediction. Sections 7 and 8 give the complete feature definitions.

1.2 Comparisons in this report

The main experiment divides the 225 queries into 135 training, 45 validation and 45 test queries. The test partition is shared by both methods. We compare their selected operating points and also examine fixed-model rankings at identical cutoffs.

A separate development study compares the SVM with BM25 and TF-IDF cosine. It uses only the 180 training and validation queries. Regression checks cover the calculations and data flow; retrieval quality is measured from the judged result lists. The two sets of evidence are reported separately.

2 Collection, judgments and query identifiers

The collection contains 1,400 document records, 225 queries and relevance judgments. Each document starts with .I and has title (.T), author (.A), bibliography (.B) and body (.W) sections. Queries contain .I and .W sections. Each judgment gives a query ID, a document ID and a relevance grade.

Table 1: Statistics of the supplied Cranfield collection.

Quantity	**Value**
Documents	1400
Queries	225
Judgment records	1837
Relevant judgments, grades 1-4	1612
Grade -1 records	225
All-field tokens	256,861
Distinct terms	9,422
Mean stream length	183.472
Empty streams	471, 995

The counts above use all four fields after tokenization. Documents 471 and 995 have empty streams. We retain both documents and define their length-normalized features and cosine values as zero.

2.1 Relevance conversion

Grades 1 through 4 become positive labels; grade 1 is the strongest relevance grade. Grade -1, 0, 5 and unjudged pairs become negative labels. The supplied file contains 225 grade -1 records and no grade 5 records. The SVM learns binary relevance rather than the numerical grade.

This conversion groups different degrees of relevance together. It also counts unjudged documents as negative, although some may be useful. Precision and recall therefore refer to the known relevant sets in the supplied judgments. The example tables show the original grades where available.

2.2 Aligning queries with judgments

The query IDs need particular care. The judgments use each query's one-based position in the original query file, while the source .I values have gaps. Source IDs 1, 2, 4 and 8 consequently map to judgment IDs 1, 2, 3 and 4. Using the source IDs directly would attach some queries to the wrong judgments.

Training uses the position-based IDs and records their correspondence with the source IDs. Saved test queries already contain the position-based IDs. New query files preserve explicit IDs. Section 17 shows how to choose the ID convention when evaluating the original queries.

3 Implementation overview

The two rankers share document preparation and query processing. Their learning stages differ: one fits four field weights, while the other fits a linear function over 22 features. Figure 1 follows these stages through model selection, test evaluation and later prediction.

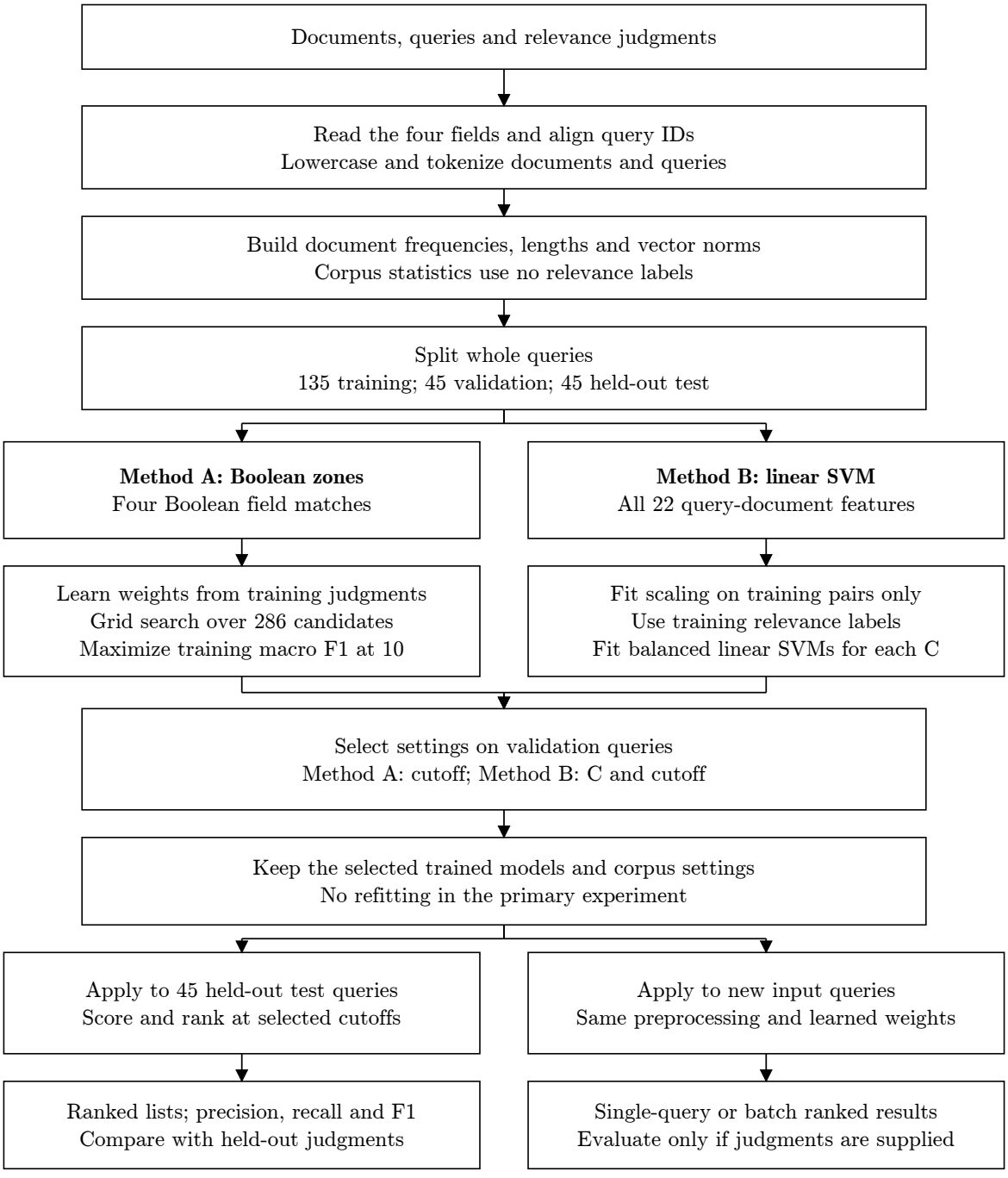


Figure 1: Training, model selection, held-out evaluation and model reuse.

3.1 Learning the two ranking functions

Preparation stores a term set for each field and a stream containing term frequencies. It also calculates document lengths, document frequencies and vector norms. Document IDs remain in a fixed order throughout feature calculation and scoring.

Method A evaluates candidate field weights by ranking the training queries and measuring macro F1. Method B pairs each training query with every document, calculates the 22 features and assigns a binary label from the judgments. Its scaler is fitted on those training pairs before the SVM coefficients are learned.

Validation selects the cutoff for both methods and C for the SVM. The primary experiment keeps the models fitted on the training partition. It uses the test judgments after producing the final rankings. The nested baseline study has a separate fitting and selection procedure, described in Section 14.

3.2 Applying the learned scores

For a new query, Method A forms the four Boolean matches and adds their weighted contributions. Method B calculates the feature vector and applies the saved linear decision function. Prediction uses the saved preprocessing settings and checks the feature order before scoring.

Both methods sort by decreasing score and break equal-score ties by increasing document ID. They then return the selected number of candidates. No new fitting is required. If judgments accompany the request, evaluation follows ranking and uses the list that was actually returned.

3.3 Keeping the experiment reproducible

We record the query partitions, parameters, selected settings, rankings and evaluation counts for each experiment. These records supply the tables and examples in the report and allow the calculations to be checked. Sections 16 and 17 give the reproduction and query commands.

4 Parsing, preprocessing and corpus statistics

The parser reads field markers, joins multiline text and preserves record IDs. Missing fields remain empty. Repeated sections of the same field are concatenated, which preserves content that would otherwise be lost when reading the supplied collection.

Parsing rejects nonpositive or repeated record IDs, text outside a field and incomplete judgment rows. Judgment validation also checks the grade range and duplicate query-document pairs. These checks catch ID and formatting errors before model fitting.

4.1 Shared tokenization

We lowercase text and extract tokens with [a-z0-9]+. Punctuation separates tokens, and numbers are retained. The stream keeps repeated occurrences for the frequency features. Boolean field matching uses sets, since repeated terms do not change whether a field matches.

This preprocessing leaves common words in the stream and keeps different word forms separate. Common words can trigger many field matches, while related forms can fail to match. We have not measured alternative preprocessing choices, so the reported results use this same tokenizer throughout.

4.2 Two views of a document

Method A uses the four fields separately. Method B normally concatenates them into one stream, retaining repeated content. A body-only option changes the SVM stream while preserving the four field sets used by Method A.

For each stream, we count term occurrences and total tokens. Document frequency counts the documents containing a term, with at most one contribution per document. Average stream length includes empty documents. An all-empty synthetic collection uses a fallback denominator of one to keep the feature calculations defined.

4.3 Query-independent statistics and edge cases

IDF, average length and document norms are calculated from the searchable corpus without relevance labels. These statistics describe the collection available to the ranker. The supervised scaler and model weights, in contrast, are fitted using training pairs only.

Both models return an empty list for an empty processed query. Method A also returns no results when every query term is unknown. The default SVM can still rank such a query through its length feature; an optional lexical-only policy requires a matching term. Section 13 describes this behavior in more detail.

5 Method A: Boolean field matches and ranking

Method A uses four binary values in the order title, author, bibliography and body. One weight is learned for each field, and the same four weights apply to every document in the collection.

5.1 Matching rule

Let Q contain the distinct processed query terms. In the reported experiment, a field matches when it contains at least one term from Q. A conjunctive option instead requires all terms in the same field. An empty query has no matching fields.

For a match vector [1, 0, 1, 1], the title, bibliography and body contribute to the score. The author field contributes zero. The score is calculated as follows:

```
score_A(q, d) = w_title * match_title
              + w_author * match_author
              + w_bibliography * match_bibliography
              + w_body * match_body
```

The weights are nonnegative and sum to one, so the score lies between zero and one. It increases when a document matches fields with larger learned weights.

Table 2: Learned Boolean field weights.

Field	Learned weight
Title	0.3
Author	0.1
Bibliography	0.2
Body	0.4

5.2 Example calculation

For heated aircraft, document 1362 has matches [1, 0, 1, 1]. Its score is 0.3 + 0.2 + 0.4 = 0.9. Document 13 scores 0.7 from a different match pattern. Section 13 lists the first five results from each saved model for this query.

The zone ranker excludes zero-score documents, orders the remaining candidates by score and returns up to k results. If fewer than k documents match, precision uses the number actually returned. Equal scores are resolved by document ID.

5.3 Limits of Boolean field matching

A Boolean field bit records presence but loses frequency, word order and the number of distinct matching terms. Documents with the same four-bit pattern receive identical scores even when their topical relevance differs. Field weights can reorder the patterns, but cannot separate documents within a pattern.

6 Learning the zone weights

We search a simplex grid with step 0.1. Each candidate is a nonnegative four-part vector summing to one. Enumerating integer allocations that sum to ten gives 286 candidates, including single-field and mixed-field choices.

Each candidate ranks the 135 training queries at cutoff 10. The objective is their mean F1, and an exact tie keeps the first enumerated candidate. The selected weights are 0.3 for title, 0.1 for author, 0.2 for bibliography and 0.4 for body. Their training macro F1 is 0.010520.

6.1 Choosing the returned list length

With those weights fixed, we evaluate cutoffs 5, 10, 15, 20 and 30 on the 45 validation queries:

Table 3: Method A validation results at the tested cutoffs.

Cutoff	Validation macro F1
5	0.002339
10	0.004191
15	0.003384
20	0.002840
30	0.005432

Cutoff 30 gives the highest validation macro F1 among the tested settings. The primary Method A test run therefore returns up to 30 positive-score documents per query.

6.2 Avoiding repeated ranking work

There are only 16 possible four-field match patterns. Some weight vectors give the same ordering and ties over these patterns. We cache their training objective when their positive-score candidates also agree. This avoids repeating an equivalent ranking calculation.

Scores are rounded to 12 decimal places to preserve the intended grid ties. Direct exhaustive checks agree with the cached learner for several grids, including the default grid.

6.3 Observed limitation

On validation queries, the largest positive-score tie group averages 872.60 documents and reaches 1,221 documents for one query. Within such a group, the document-ID tie rule determines the order. Searching a finer weight grid would still leave equal-pattern documents tied.

The learning objective uses F1 at 10, while validation selects a final cutoff of 30. The weights are therefore not optimized specifically for the returned list length. Joint selection of weights and cutoff is a possible follow-up experiment.

7 SVM features: query coverage and raw frequencies

Each query-document pair has 22 features in the assignment's order. The formulas below fix the statistical conventions used in this experiment. The feature order is retained with the learned parameters and reused at prediction time.

Let Q be the distinct query terms after tokenization. Let c(t) be the number of times term t occurs in the query, f(t,d) its raw frequency in the selected document stream, L the stream length, N the number of documents and df(t) the number of streams containing t. The smoothed IDF used for TF-IDF features is:

```
idf(t) = 1 + ln((N + 1) / (df(t) + 1))
```

Table 4: Query, coverage and raw-frequency features.

Position	Feature	Definition
1	Query term number	Total query tokens, including repetitions.
2	Query term ratio	Distinct query terms found in the stream divided by the number of distinct query terms.
3	Stream length	Total processed tokens in the chosen stream.
4	IDF	Sum of smoothed IDF over distinct query terms that occur in the stream.
5	TF sum	Sum of raw frequencies over Q.
6	TF minimum	Smallest raw frequency over Q, including zeros.
7	TF maximum	Largest raw frequency over Q.
8	TF mean	Arithmetic mean of raw frequencies over Q.
9	TF variance	Population variance of raw frequencies over Q.

Frequency summaries range over every distinct query term, with zero for an absent term. For example, omitting one query term makes the minimum TF zero even if another term occurs many times. Variance is population variance, divided by the number of distinct query terms.

7.1 Worked frequency example

For query alpha alpha beta, suppose a document stream is alpha alpha beta gamma. The query has three tokens but two distinct terms. Its raw frequency vector over Q is [2, 1]. Features 1, 2 and 3 are 3, 1 and 4. The TF sum, minimum, maximum, mean and population variance are 3, 1, 2, 1.5 and 0.25. IDF values depend on the rest of the collection and cannot be assigned from this document alone.

Repeated query terms contribute to the query token count and cosine query vector. The TF summaries use distinct terms, so the example has one alpha frequency entry rather than two. The independent calculator checks this convention along with the other feature formulas.

Query length is constant across documents for a given request. Its coefficient alone cannot change that request's document order, although it affects classification margins across queries of different lengths. We retain it as part of the required feature vector.

8 SVM features: normalization and retrieval scores

The remaining features cover normalized TF, TF-IDF and three retrieval scores. For each frequency group, the summary order is sum, minimum, maximum, mean and population variance.

Table 5: Normalized frequencies and retrieval-score features.

Positions	Feature group	Definition
10-14	Normalized TF	Five summaries of f(t,d) / L over Q.
15-19	TF-IDF	Five summaries of f(t,d) * idf(t) over Q.
20	Boolean model	One if every distinct query term occurs in the stream; otherwise zero.
21	Vector space model	Cosine similarity between raw-TF times smoothed-IDF query and document vectors.
22	BM25	Sum of BM25 term contributions with k1 = 1.2 and b = 0.75.

Normalized frequencies are zero for a zero-length stream. For the frequency example in Section 7, the normalized vector is [0.5, 0.25]. Its sum is 0.75, mean is 0.375 and variance is 0.015625. Dividing the raw variance by L would be incorrect; the variance of the normalized values divides the raw variance by L squared.

8.1 Cosine similarity

Document weights use raw term frequency multiplied by the smoothed IDF from Section 7. Query weights use c(t) multiplied by the same IDF. The dot product involves query terms, but the document norm includes its full vocabulary. Restricting that norm to matched query terms would overstate similarity and change the feature.

```
document_weight(t) = f(t,d) * idf(t)
query_weight(t) = c(t) * idf(t)
cosine = dot(document_weights, query_weights)
         / (document_norm * query_norm)
```

Cosine is zero when either norm is zero. An unknown query term has document frequency zero and contributes no document weight. It still contributes to the query norm, reducing cosine similarity when the rest of the query has a partial match.

8.2 BM25

BM25 uses its own IDF expression rather than reusing the smoothed TF-IDF expression. The sum includes each distinct query term once. Let average˙L be the mean stream length across the corpus.

```
bm25_idf(t) = ln(1 + (N - df(t) + 0.5) / (df(t) + 0.5))
length_norm = 1.2 * (0.25 + 0.75 * L / average_L)
bm25 = sum(bm25_idf(t) * f(t,d) * 2.2
           / (f(t,d) + length_norm) for t in Q)
```

An empty query has zero-valued features apart from stream length, and the ranking interface returns no results for it. Empty streams and absent terms produce finite feature values. The tests cover these cases for both stream options.

9 Linear SVM training and exported weights

The SVM training set contains 189,000 pairs: 135 queries paired with 1,400 documents. Only 991 pairs are positive. Each feature is standardized using the training-pair mean and population standard deviation; constant columns use scale one.

We fit a linear SVM with balanced class weights, squared hinge loss and L2 regularization. The solver uses primal optimization, seed 42 and a limit of 20,000 iterations. Fitting stops if it fails to converge. Class balancing gives the rare positive examples more influence in the training objective.

Table 6: Original-unit and standardized SVM coefficients.

Feature	Original-unit weight	Standardized weight
f1	-0.0194058	-0.135497
f2	+0.463602	+0.057099
f3	-0.000115603	-0.010393
f4	-0.0564386	-0.276684
f5	-0.0114176	-0.234726
f6	+0.198982	+0.001650
f7	-0.0249545	-0.231572
f8	-0.057942	-0.071641
f9	+0.00804073	+0.237202
f10	+2.10505	+0.166341
f11	+45.9213	+0.005458
f12	+1.37187	+0.045029
f13	-52.5741	-0.246634
f14	+17.1219	+0.005475
f15	+0.00714147	+0.175562
f16	+1.39056	+0.017122
f17	-0.00102308	-0.009392
f18	+0.134272	+0.196365
f19	-0.00306328	-0.095344
f20	-7.75515	-0.035677
f21	+3.60034	+0.152649
f22	+0.135135	+0.453768

The selected primary model uses C = 0.1 and cutoff 5. Its standardized intercept is -0.494632; its original-unit intercept is -0.291903. Original-unit coefficients are obtained by dividing each standardized coefficient by its training scale and adjusting the intercept by the means. Thus score˙B = sum(weight[j] * feature[j]) + intercept reproduces the solver's decision value without loading scikit-learn during prediction.

Ranking uses the continuous decision value. The large original-unit coefficients for normalized TF partly reflect the small units of those features. Since the inputs are correlated, individual coefficient signs and magnitudes should be interpreted in the context of the fitted vector. The SVM optimizes a pointwise classification loss.

10 Experiment plan and evaluation

We use a query-level 60/20/20 split with seed 42. All document pairs for one query stay in the same partition. This keeps the training and evaluation queries disjoint while allowing both models to search the complete collection.

Table 7: Primary query partitions and binary pair counts.

Partition	Queries	Pairs	Relevant	Other
Train	135	189,000	991	188,009
Validation	45	63,000	280	62,720
Test	45	63,000	341	62,659

The selection objective is validation macro F1. Method A chooses among five cutoffs after learning its weights. Method B tries C values 0.01, 0.1 and 1 with the same cutoff set. Ties retain the smaller C or cutoff. The best validation setting for each C is shown below.

Table 8: Best primary validation setting for each SVM C.

C	Best cutoff	Validation macro F1
0.01	5	0.241029
0.1	5	0.247378
1	5	0.247378

After selection, we keep the models fitted on the 135 training queries. The 45 test queries provide the primary comparison. Corpus statistics use no judgments; scaling and fitted weights use training pairs; C and cutoff use validation performance. The common-cutoff results in Section 11 describe the fixed models and do not change their selected settings.

10.1 Precision, recall and F1

For each query, hits counts returned documents in its known relevant set, returned is the actual list length and relevant is the size of that set. We calculate:

```
precision = hits / returned
recall = hits / relevant
F1 = 2 * precision * recall / (precision + recall)
```

A zero denominator gives zero. Precision uses the actual list length, including when the zone model returns fewer than k candidates. Duplicate document IDs are rejected.

Macro averages give each query equal weight and include zero-hit queries. Macro F1 is the average of the query F1 values. Micro measures pool the hit, returned and relevant counts before applying the formulas. We report both because relevant-set sizes differ between queries. Evaluation uses the same lists written to the ranking output.

11 Test results and discussion

The primary test partition has 45 queries and 341 positive judgments. We use the validation-selected cutoffs: 30 for Boolean zones and 5 for the SVM.

Table 9: Primary held-out precision, recall and F1.

Method	Average	k	Precision	Recall	F1
A	macro	30	0.012593	0.075582	0.019683
A	micro	30	0.012593	0.049853	0.020106
B	macro	5	0.315556	0.260766	0.270154
B	micro	5	0.315556	0.208211	0.250883

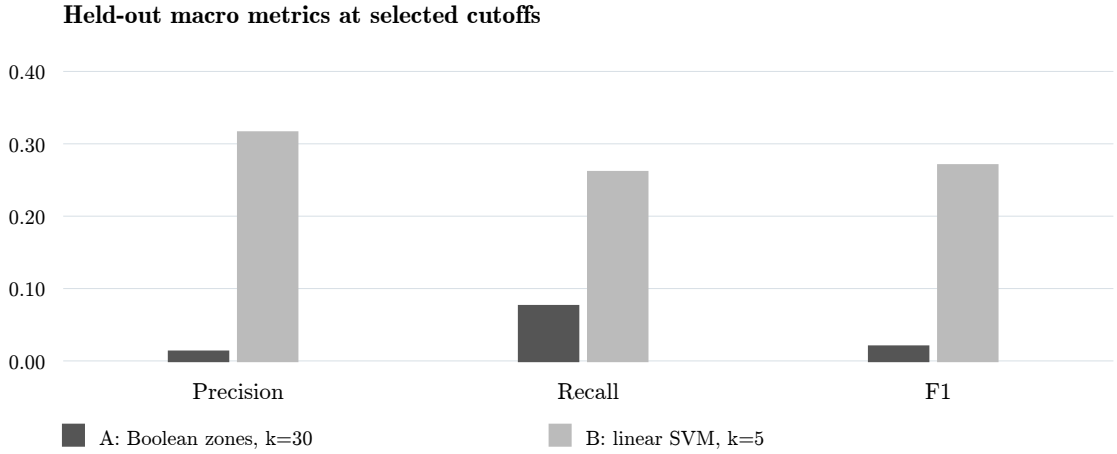


Figure 2: Held-out macro metrics at the selected cutoffs.

The zone model retrieves 17 judged relevant documents in 1,350 returned positions. The SVM retrieves 71 in 225 positions. Its precision and recall are both higher, although its lists are shorter. It has zero hits on 16 queries, compared with 30 for the zone model. Its per-query F1 is higher on 29 queries, equal on 13 and lower on 3.

11.1 Comparing identical cutoffs

The next table fixes the learned weights and truncates both models at the same cutoff. It controls list length when comparing their descriptive test behavior.

Table 10: Held-out macro metrics at identical cutoffs.

k	A P	A R	A F1	B P	B R	B F1
5	0.0089	0.0067	0.0074	0.3156	0.2608	0.2702
10	0.0111	0.0130	0.0112	0.2311	0.3686	0.2662
15	0.0119	0.0215	0.0146	0.1881	0.4389	0.2455
20	0.0100	0.0228	0.0133	0.1500	0.4570	0.2118
30	0.0126	0.0756	0.0197	0.1126	0.5063	0.1749

At the common cutoff of 5, macro F1 is 0.2702 for the SVM and 0.0074 for Boolean zones. The difference therefore persists when list length is held equal. For the SVM, increasing k from 5 to 30 raises recall from 0.2608 to 0.5063, but lowers precision from 0.3156 to 0.1126 and F1 from 0.2702 to 0.1749. The primary operating point remains the one chosen on validation queries.

11.2 Why the two models differ

The main weakness of Boolean scoring is the number of documents that share a match pattern. A common word can make a field match just as readily as a topic-specific word. Once the four bits are fixed, neither additional matching terms nor higher frequency changes the score. The large validation tie groups explain why the document-ID rule has substantial influence on the result lists.

The SVM has more information with which to separate these documents. Coverage, normalized frequency, cosine and BM25 can vary even when the Boolean field bits are identical. The stronger results are consistent with this extra discrimination. We have not run feature ablations, so these experiments do not isolate the contribution of any one feature.

11.3 Limits of the result and follow-up experiments

The SVM's macro F1 exceeds the zone model's by about 0.2505 on this split, but 16 zero-hit queries remain. The strong result for query 223 in Section 12 is only one example. Further query-level analysis is needed to determine how often limited lexical overlap or missing judgments account for the failures.

Training also differs from the evaluation objective. The SVM minimizes a classification loss, while settings are selected by list-level F1. Method A learns weights at cutoff 10 and returns up to 30 results after validation. These mismatches may limit the quality of the selected rankings. They are reasonable targets for further experiments, rather than evidence that the present settings are optimal.

A follow-up study could learn zone weights jointly with cutoff, compare stopword and stemming choices, and vary SVM regularization or class weighting. These choices should be selected using development queries. The current report contains no measured gain from those alternatives. Once a test set has informed repeated development, a fresh unseen query set is needed for an independent final check.

12 A judged test-query example

Query 223 is a held-out query. Its source ID is 356, while 223 is the identifier used by the supplied judgments. Its text is:

papers on shear buckling of unstiffened rectangular plates under shear .

The SVM returns three judged relevant documents and two unjudged documents for this query. The complete result list makes the metric calculation easy to check.

12.1 Method B: all five returned documents

Table 11: All five SVM results for test query 223.

Rank	Document	Score	Qrel grade
1	400	2.996310	3
2	1399	2.613746	Unjudged
3	1387	1.879368	3
4	1400	1.874723	Unjudged
5	1398	1.801843	3

Documents 400, 1387 and 1398 are positive judgments. Documents 1399 and 1400 are unjudged and count as negative for evaluation. With three hits, five returned documents and four known relevant documents, precision is 3/5, recall is 3/4 and F1 is 2/3.

12.2 Method A: first five of its 30 returned documents

Table 12: First five Boolean-zone results for test query 223.

Rank	Document	Score	Qrel grade
1	240	1.000000	Unjudged
2	2	0.900000	Unjudged
3	3	0.900000	Unjudged
4	17	0.900000	Unjudged
5	19	0.900000	Unjudged

Method A returns 30 documents for this query; the table displays only its first five. None of the 30 is a positive judgment. Its first document matches every field and scores one, while the equal scores below it are ordered by document ID. The Boolean matches are therefore insufficient to produce a useful ranking for this request.

12.3 Reading the example

The two score scales differ, so a value such as 2.996310 cannot be compared numerically with a zone score of 1.0. The comparison is between the returned document orders and their judged relevance.

The aggregate evaluation includes all queries, including the 16 zero-hit SVM cases. This example illustrates the scoring behavior but does not stand in for that aggregate result.

13 A new-query result and the output format

For a new-query example, we use heated aircraft with query ID 901 and request five results from each saved model. No judgments accompany this request, so the example shows rankings and scores without assigning precision, recall or F1.

These are the actual saved-model outputs. Section 17 gives the corresponding single-query and batch commands.

13.1 SVM output

```
901 Q0 1268 1 2.1770113049064359 PA3_B
901 Q0 51 2 2.057738789212511 PA3_B
901 Q0 13 3 1.9510417954324248 PA3_B
901 Q0 100 4 1.4537703676285045 PA3_B
901 Q0 154 5 1.435145239384978 PA3_B
```

13.2 Boolean-zone output

```
901 Q0 1362 1 0.90000000000000002 PA3_A
901 Q0 13 2 0.69999999999999996 PA3_A
901 Q0 47 3 0.69999999999999996 PA3_A
901 Q0 51 4 0.69999999999999996 PA3_A
901 Q0 66 5 0.69999999999999996 PA3_A
```

The six columns are query ID, Q0, document ID, rank, score and run tag. Rank starts at one for each query. Scores descend, with document ID breaking ties. Scores retain enough digits for numerical reproduction. The query ID identifies the request and does not enter the scoring function.

The SVM ranks document 1268 first, while the zone model ranks document 1362 first. Both use the same query and collection. Here k is explicitly set to 5; the saved zone-model default remains 30.

13.3 Empty or entirely unknown requests

Empty processed queries return no results. For a nonempty query made entirely of unknown terms, the zone model has no candidates. The default SVM can return length-dependent scores, including negative values. The lexical-only candidate policy instead requires a query-term match. This policy is retained with the model and used consistently in evaluation and prediction.

The run tags are PA3˙A and PA3˙B. An empty result has no ranking lines, but its query still contributes to an evaluation generated from the input queries. This keeps empty requests in the macro averages.

14 Baseline comparison on development queries

We also compare the SVM with BM25 and TF-IDF cosine to assess how much it adds to standard retrieval scores. This study uses the 180 original training and validation queries. The 45 primary test queries are excluded.

Three outer folds hold out 60 whole queries each. Within the other 120 queries, an inner split uses 96 for fitting and 24 for selection. SVM C is selected from 0.01, 0.1 and 1. Every integer cutoff from 1 to 30 is considered. Each baseline selects its own cutoff on the same inner validation queries. The chosen SVM is then refitted on all 120 outer training queries with a new scaler before predicting the outer holdout. Seed 42 fixes the splits.

Table 13: Development out-of-fold macro results.

Method	Precision	Recall	F1
SVM	0.285880	0.333460	0.272459
BM25	0.288836	0.309396	0.264053
cosine	0.271254	0.313238	0.249020

Each development query has one outer-fold prediction. The reported macro averages summarize all 180 of these predictions. Their population and fitting procedure differ from the primary test experiment, so they are reported as a separate comparison.

Table 14: Settings selected within each development outer fold.

Fold	SVM C	SVM k	BM25 k	Cosine k
1	0.01	8	6	6
2	0.01	6	5	4
3	0.01	6	7	11

14.1 Paired differences and uncertainty

For each query, we subtract the baseline F1 from SVM F1. We use 10,000 paired bootstrap samples for a 95% percentile interval and a two-sided paired sign-flip test. Holm adjustment accounts for the two baseline comparisons.

Table 15: Paired F1 differences and adjusted significance tests.

Comparison	F1 difference	95% interval	Holm p
SVM minus BM25	+0.008406	[-0.009651, +0.025943]	0.3566
SVM minus cosine	+0.023439	[+0.004694, +0.041723]	0.0328

SVM has the highest mean F1, but its advantage over BM25 is small: 0.008406. The interval includes zero and the adjusted p-value is 0.3566. The data therefore support similar performance for these two methods under this protocol, rather than an established SVM improvement. Against cosine, the F1 difference is 0.023439 and the adjusted p-value is 0.0328.

These intervals describe the paired query differences for the fitted folds. Outer training sets overlap, so they do not capture all training-set variation. One fold assignment and binary judgments are further limits. Taken together, the experiments show a large improvement over the Boolean model on the primary split, but a much less certain advantage over BM25 in the development study.

15 Regression checks and computational cost

We used 41 regression tests to check parsing, feature formulas, learning, saved-model prediction and evaluation. The recorded full run completed with no failures, errors or skips. A separate query audit within the same suite checks ranking consistency and input handling.

Table 16: Coverage of the consolidated regression checks.

Area	Evidence from the checks
Parsing	An independent reader agrees on all 1,400 documents, including repeated field sections.
Features	35,200 values are checked independently across 25 documents, 32 queries, two streams and 22 features.
Zone learning	Cached grid objectives match direct exhaustive evaluation, including pattern ties and positive-score filtering.
SVM scores	Exported original-unit scores agree with the fitted solver's decision function.
Saved results	Every test query is scored against all 1,400 documents; rankings and metrics agree with the saved files.
Label isolation	Changing final-test labels leaves the learned models, cutoffs and exported rankings unchanged.
Query inputs	Positive IDs, BOM files, malformed records, aliases and prediction from another directory are checked.
Validation study	Fold separation, inner selection and paired uncertainty calculations are checked on controlled examples.

The audit checks 360 case, whitespace, punctuation, top-k prefix and unique-ID properties across both models and all 45 validation queries. It also tests invalid query IDs and BOM-encoded query records.

15.1 Full reproduction

A full reproduction run trains both models in a temporary directory. All eight experiment files match the supplied outputs byte for byte, and dataset hashes remain unchanged. The largest discrepancy between standardized-coordinate and exported SVM scores is about 2.22e-15.

15.2 Time and memory

With D documents and m distinct query terms, feature extraction takes O(D*m) after corpus statistics are available. Linear scoring takes O(D*22), and sorting takes O(D*log D). Zone fitting searches 286 candidates and reuses equivalent rankings. SVM fitting time depends on the solver and data.

The primary run caches 6,930,000 feature values, along with scaled training rows and corpus statistics. Python objects require more memory than a packed numeric array. Training and cross-validation should therefore run sequentially on a memory-limited machine. Batching feature extraction would be a practical improvement for larger collections.

16 Running the programs

Run the following PowerShell commands from the project root. A backtick continues a command onto the next line. The project requires Python 3.10 or later. The recorded experiments used Python 3.12.2 and scikit-learn 1.7.2; solver-version changes may affect exact numerical reproduction.

16.1 Install the training dependency

```
python -m pip install -r requirements.txt
```

Scikit-learn is the training dependency, with the supported range recorded in requirements.txt. Pip installs its transitive dependencies. The Cranfield collection is already included, so no corpus download is needed.

16.2 Reproduce the main experiment

```
python src/train.py
```

The default run reads the supplied collection and writes learned weights, settings, partitions, test queries, rankings and evaluation to results/. Use a separate output directory for an alternative experiment:

```
python src/train.py --output results/body_experiment --stream body
```

Training options include --seed, --grid-divisions, --zone-k, --zone-match, --stream, --svm-candidates, --cutoffs and --c-values. Their defaults reproduce the primary experiment. Select alternative settings using development queries and record them with the resulting model.

16.3 Run the consolidated tests

```
python -B tests/run_tests.py
python -B tests/run_tests.py --retrain
```

The first command runs the regression suite and query audit. Adding --retrain also fits both models and compares the eight experiment outputs. The script recreates results/test˙results.txt, results/output˙verification.txt and results/regression˙audit.txt. A full reproduction also writes results/retraining˙test.txt.

16.4 Run the additional validation study

```
python -B src/validate.py
```

This writes the development comparison to results/cross˙validation.txt without changing the primary model or test rankings. Run it separately from full training if memory is limited. Each script accepts --help for its supported arguments.

17 Running queries and reading the results

Prediction loads the saved weights and document collection. Method names A and zone select Boolean scoring; B and svm select the SVM. Omitting --top-k uses the model's saved cutoff.

```
python src/predict.py --method svm --query "heated aircraft" ‘
  --top-k 5 --output results/search_svm.txt
python src/predict.py --method zone --query "heated aircraft" ‘
  --top-k 5 --output results/search_zone.txt
```

17.1 Batch files

Batch prediction accepts Cranfield records, ID-tab-text rows or one plain query per line. Save the following Cranfield example as queries.txt:

```
.I 901
.W
heated aircraft
.I 905
.W
shear buckling rectangular plates
```

```
python src/predict.py --method svm --queries queries.txt ‘
  --output results/batch_svm.txt
```

The explicit IDs 901 and 905 are preserved. Tab-separated input must contain actual tab characters. Plain lines use physical line numbers as IDs, with blank lines skipped. UTF-8 files with a BOM are accepted. Duplicate or nonpositive explicit IDs are rejected.

17.2 Evaluate saved test queries

```
python src/predict.py --method svm --queries results/test_queries.txt ‘
  --qrels data/cran/cranqrel --output results/replayed_test.txt
```

The command also writes results/replayed˙test.evaluation.txt. Saved test queries already have judgment-aligned IDs. For the original data/cran/cran.qry, add --query-id-mode ordinal when using the original judgments. New query IDs require matching judgments of their own.

17.3 Use from Python or load an alternate experiment

```
import sys
sys.path.insert(0, "src")
from predict import load_models, rank_query
models = load_models()
print(rank_query(models, "heated aircraft", method="B", top_k=5))
```

Use --weights results/body˙experiment/weights.txt to load an alternate model. The --documents option selects a collection, and --query-id labels a single-query request. Default collection and weight paths are resolved from the source tree, allowing prediction from another working directory.